

CampusLab — Arquitectura Cloud Native segura

Federación Microsoft Entra ID + Amazon Cognito · API Gateway · BFF · EC2/Docker

Williams Rivas

Ingeniería en Informática — DSY1107 Desarrollo Cloud Native I

Caso 2 · Septiembre 2026

Contexto y objetivo

- CampusLab: reserva de laboratorios y equipos académicos para una red de instituciones.
- Objetivo: sistema **funcional y seguro** en la nube, con identidad corporativa de Microsoft.
- Frontend React · Backend Spring Boot · Cognito federado a Entra · API Gateway · EC2/Docker.
- Regla de oro: **JWT** → **API Gateway** → **BFF** → **microservicio de dominio**.

Arquitectura

Componentes

- Frontend React (MSAL + Cognito PKCE)
- Amazon Cognito (IDaaS, federado a Entra)
- API Gateway HTTP API (JWT authorizer)
- BFF Spring Security (roles, CORS)
- Microservicios catalog + bookings

Navegador (React)

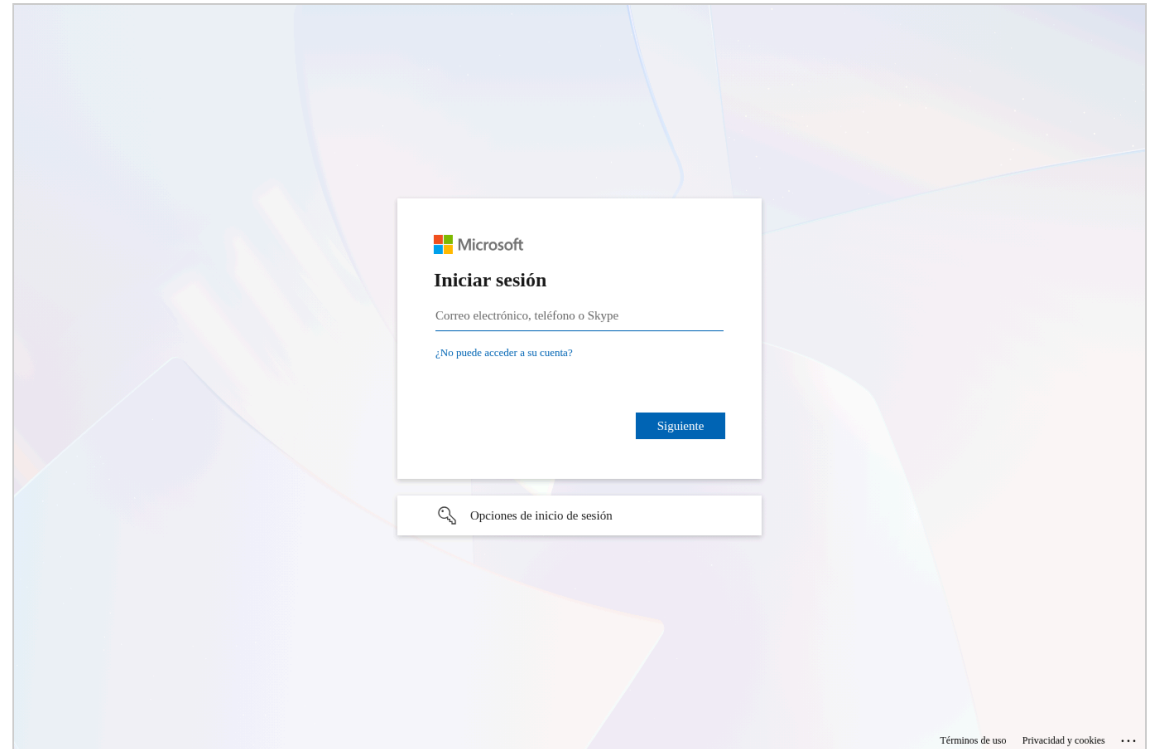
- | 1) Login Microsoft (Entra)
- | 2) Auth Code + PKCE (Cognito)
- | 3) access token (JWT)



API Gateway —JWT→ BFF (EC2)
401 sin token | rol + CORS
| ▶ catalog
| ▶ bookings

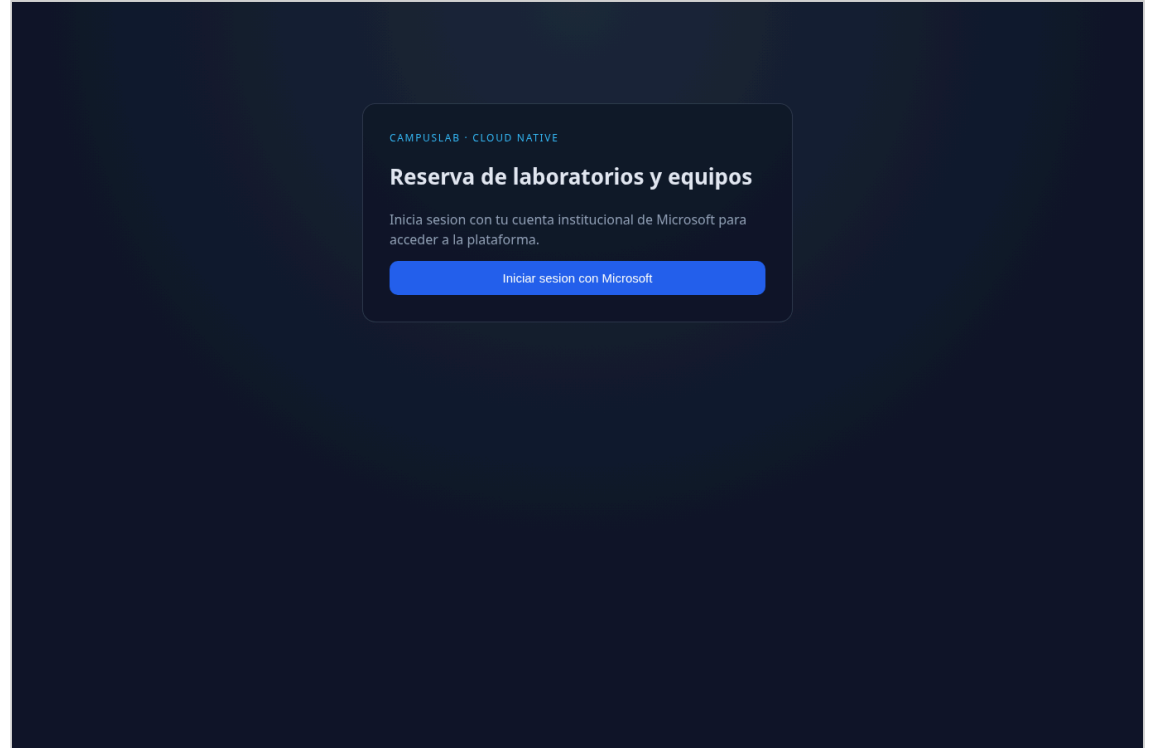
Identidad: Entra ID + Cognito (federación)

- App Registration en Entra ID (tenant WRIVAST).
- Cognito: proveedor OIDC → EntraID; grupos Admin/Operador.
- App Client público con Authorization Code + PKCE.
- El login se **delega en Microsoft**; Cognito emite el token.



Login OIDC — Authorization Code + PKCE

- El cliente genera `code_verifier` y `code_challenge` (S256).
- App Client público: **sin secreto** en el navegador.
- Token con `token_use=access`, `client_id` y `cognito:groups`.
- El frontend adjunta `Authorization: Bearer` en cada llamada.



API Gateway y seguridad

Control	Configuración real
Authorizer	JWT · Issuer cognito-idp.us-east-1.amazonaws.com/us-east-1_dBOQeniQq
Audience	6r3af8ucc96okla0ujph9pru4a (client_id)
CORS	Origin http://localhost:5173 · GET,POST,PUT,DELETE,OPTIONS
Roles (BFF)	cognito:groups → ROLE_ADMIN / ROLE_OPERADOR

- API Gateway valida en el borde; el BFF revalida y controla el rol por endpoint.

Evidencia del CRUD real

```
GET /api/catalog/resources sin token -> 401 (API Gateway)
GET /api/catalog/resources token Admin -> 200 [5 productos]
POST /api/catalog/resources Admin -> 201 {"id":6,"nombre":"Laboratorio de IoT"}
PUT /api/catalog/resources/1 Admin -> 200
POST /api/catalog/resources usuario NO Admin -> 403 (BFF: rol sin permiso)
POST /api/bookings Admin (valido) -> 201 {"estado":"SOLICITADA"}
POST /api/bookings rut/nombre invalidos -> 400 (validacion de esquema)
PUT /api/bookings/2/status Admin -> 200 {"estado":"EN_USO"}
DELETE /api/bookings/1 Admin -> 204
```

Real contra el Invoke URL del API Gateway, tokens reales de Cognito.

Despliegue en EC2 con Docker

- EC2 Amazon Linux 2023 (t3.medium) — i-0448753bab6e8173b — 3.237.97.7
- Docker Compose: catalog, bookings y BFF.
- Security Group: solo 22, 8080 y 8090.

```
$ sudo docker compose ps
campuslab-catalog    Up      8081
campuslab-bookings   Up      8082
campuslab-bff        Up      0.0.0.0:8080->8080
$ curl http://3.237.97.7:8080/actuator/health -> {"status":"UP"}
```

Conclusiones y alcance

- Arquitectura Cloud Native **funcional y segura**, desplegada de extremo a extremo.
- Identidad federada a Microsoft; JWT validado en API Gateway y BFF; roles y CORS.
- CRUD con validación de esquema (nombre/rut/fecha): 200/201/204/400/401/403.
- Fuera de alcance: Kafka, KPIs/analytics, auditoría por streaming y **AWS Lambda**.
- Repositorio: github.com/WilliamsRT93/campuslab-caso2